BASIC OF JAVA PROGRAMMING

SCENARIO BASED QUESTIONS

When I started preparing for interviews years back, I realized something: It's one thing to know the definitions and theory, but when an interviewer asks you to apply concepts in a real-world scenario, that's when most candidates freeze.

That's exactly why I designed this Scenario-Based Questions section. These are not random coding puzzles, each scenario is tied to practical use cases like e-commerce, banking, event management, or simple utilities you can easily relate to.

Here's how I suggest you use them:

- **Read the problem carefully**: Understand what real-world situation it's simulating.
- **Think through the logic in plain English first**: Don't jump into code immediately.
- **Study the reasoning in the answers:** I've explained *why* I used a particular loop, data structure, or method, so you see the thought process.
- **Try coding without looking**: Then compare your version with the solution.
- **Experiment with tweaks**: change inputs, extend the logic, or add validations. That's where real learning happens.

If you practice these well, you'll build the habit of turning interview questions into simple problem-solving exercises, and you'll never get stuck when asked, *"Okay, can you write me a quick program for this case?"*

Take these scenarios seriously. They're like mini-mock interviews hidden inside this guide.
And remember: the goal isn't just to answer... it's to **explain clearly and confidently, step by step**, just like in a real interview.

Good luck, let's dive into the scenarios.

#	Scenario (Short)	Type	Level	Role Focus
1	Validate age is positive, name not blank	Validation	L1	Backend Core
2	Mark roll number as even/odd	Control Flow	L1	Backend Core
3	Categorize membership by age	Conditional Logic	L1	Backend Core
4	Map menu choice (1→Pizza, 2→Burger, 3→Salad)	Switch / Mapping	L1	Backend Core
5	Sum daily savings ₹1–₹20	Loop / Math	L1	Backend Core
6	Validate feedback text not empty	Validation	L1	Backend Core
7	Stop processing when code 105 found	Loop Control	L1	Backend Core
8	Skip question 5 while looping 1–10	Loop / Continue	L1	Backend Core
9	Define constant role codes (ADMIN, CUSTOMER)	Constants / Final	L1	Backend Core
10	Temporary variable scope inside if-block	Variable Scope	L1	Backend Core
11	Check if number positive, negative, or zero	Conditional Logic	L1	Backend Core
12	Countdown 5→1, then “Time’s up!”	Loop / Output	L1	Backend Core
13	Check divisibility by 4 & 6	Conditional Logic	L1	Backend Core
14	Print each character of a name	Loop / String	L1	Backend Core
15	Print month name from number (1–12)	Mapping / Validation	L1	Backend Core
16	Validate password length ≥8	Validation	L1	Backend Core
17	Compute factorial n!	Loop / Math	L1	Backend Core
18	Sum all even numbers 2–20	Loop / Optimization	L1	Backend Core
19	Output square of input number	Math / Casting	L1	Backend Core
20	Countdown from user input	Loop / Timing	L2	Backend Core
21	Trigger actions per order status (PLACED→DELIVERED)	Control Flow / Enum	L2	Backend Core
22	Reverse a sentence efficiently	String Manipulation	L2	Backend Core

23	Find highest & lowest temperatures	Array / Loop	L2	Backend Core
24	Validate email contains “@” and ends with “.org”	Validation	L2	Backend Core
25	Compute exponentiation (e.g., 5^3)	Math / Power	L2	Backend Core
26	Convert numeric string to int safely	Exception Handling	L2	Backend Core
27	Sum 2D sales data	Nested Loops	L2	Backend Core
28	Display Fibonacci sequence up to n	Iteration / Sequence	L2	Backend Core
29	Check if word is palindrome	String Logic	L2	Backend Core
30	Simulate basic login validation	String Comparison	L2	Backend Core
31	Role-based access control (Admin/User)	Conditional Access	L2	Backend Core
32	Compute GCD of two integers	Algorithm / Math	L2	Backend Core
33	Count frequency of characters	Data Structure / Map	L2	Backend Core
34	Check if number is prime	Math / Algorithm	L2	Backend Core
35	Sort integer array ascending	Sorting	L2	Backend Core
36	Handle large factorial (BigInteger)	Overflow Handling	L3	Advanced Backend
37	Remove duplicates from integer array	Data Structure / Set	L2	Backend Core
38	Format list into comma-separated string	String / Collection	L2	Backend Core
39	Raise square matrix to power	Matrix / Math	L3	Advanced Backend
40	Merge two sorted arrays (no duplicates)	Two-pointer / Array	L3	Backend Core
41	Quiz: loop until correct answer	Loop / Input	L2	Backend Core
42	Compute average safely (avoid overflow)	Math / Precision	L2	Backend Core
43	Find longest book title	String / Null Safety	L2	Backend Core
44	Multiplication table (1–10 grid)	Loop / Formatting	L1	Backend Core
45	Count vowels in sentence	String / Loop	L1	Backend Core

46	Find second highest distinct score	Array / Sorting	L2	Backend Core
47	Check if two words are anagrams	String / Sorting	L3	Backend Core
48	Stop stream if reading >100°C	Stream / Loop Control	L2	Backend Core
49	“Guess the number” game (1–100)	Loop / Random / Input	L2	Full Stack
50	Sum of digits (e.g., 90705 → 21)	Math / Loop	L1	Backend Core
51	Find second smallest distinct number	Array / Sorting	L2	Backend Core
52	Rotate array right by k positions	Array Manipulation	L3	Backend Core
53	Transpose m×n matrix	Matrix / Loop	L2	Backend Core
54	Validate 10-digit phone number	Validation / Regex	L1	Backend Core
55	Read until -1, print running average	Loop / Condition	L2	Backend Core
56	Count duplicate words in sentence	String / Map	L2	Backend Core
57	Remove vowels from string	String / Replace	L2	Backend Core
58	Sum diagonals of n×n matrix	Matrix / Loop	L2	Backend Core
59	Unique intersection of two sorted arrays	Array / Two-pointer	L2	Backend Core
60	Validate PIN (4 or 6 digits only)	Validation / Regex	L1	Backend Core

Scenario 1: Signup form validation

You're building a signup form for a small e-learning app. The form asks learners for their age (must be a whole number above zero) and a display name (should contain at least one character).

Answer:
For this case, I'd start by deciding the right data types:

- Age should be stored as an int, because it's always a whole number and doesn't need decimals.
- Display name should be stored as a String, since it can have characters, spaces, etc.

Next, I'll handle validations:

1. **Age validation**:

    - Must be greater than 0.
    - Shouldn't accept unrealistic values like negatives or zero.

```
int age = userInputAge;
if (age <= 0) {
   throw new IllegalArgumentException("Age must be greater than 0");
}
```

2. **Display name validation**:

    - Should not be null or empty.
    - At least one visible character required.

```
String displayName = userInputName;
if (displayName == null || displayName.trim().isEmpty()) {
   throw new IllegalArgumentException("Display name cannot be
empty");
}
```

3. **Putting it together:**

```
public class SignupValidator {
  public static void validate(int age, String displayName) {
     if (age <= 0) {
        throw new IllegalArgumentException("Age must be greater than
0");
     }
     if (displayName == null || displayName.trim().isEmpty()) {
        throw new IllegalArgumentException("Display name cannot be
empty");
     }
     System.out.println("Signup successful for: " + displayName);
  }

  public static void main(String[] args) {
     validate(22, "Rajat");   // Valid
     validate(-5, "John");    //Invalid age
     validate(25, "");       //Invalid name
  }
}
```

Reasoning to interviewer:

- I chose int for age because it's precise and efficient. If the business later requires fractional ages (like 2.5 years for kids), I could switch to double.
- For name, String works best since it's immutable and easy to validate with built-in methods like isEmpty() and trim().
- Validation ensures that invalid input doesn't propagate into the system, avoiding issues later (e.g., negative age showing in analytics).

When an interviewer gives you a scenario like this, they're not looking for you to dump 40 lines of code. They want to see:

1. **Your thought process** – how you decide data types, validate inputs, and handle edge cases.
2. **Clarity in explanation** – step-by-step reasoning rather than jumping straight to code.
3. **Confidence** – explaining *why* you chose int instead of double, or String instead of char[].
4. **Practical touch** – relating it to a real-world bug or use case (like negative age analytics issue).

So in an actual interview, I'd shorten the code to small snippets and focus more on talking through the design. Example answer I'd *speak out loud*:

"For age, I'd use an int because it's always a whole number. The first validation would be to ensure it's greater than zero. For display name, a String is appropriate, and I'd check that it's not null or empty, using something like trim().isEmpty() to catch blank spaces. If either check fails, I'd throw an exception or show an error message. This way, invalid data never goes further into the system. For instance, if someone enters -5 as age, I'll reject it immediately. In real-world systems, validations like this avoid problems downstream in analytics or reporting."

Scenario 2: Even/Odd Roll Numbers for Seating

In a school grading tool, you need to mark roll numbers as **"even"** or **"odd"** for seating arrangements.

Answer:

The simplest way to solve this is by using the modulus operator (**%**). In Java, num % 2 gives the remainder when divided by 2.

- If remainder = 0 → number is even
- Else → number is odd

Code Example:

```
public class SeatingArrangement {
   public static String markRollNumber(int rollNo) {
      if (rollNo % 2 == 0) {
         return "Roll No " + rollNo + " is Even";
      } else {
         return "Roll No " + rollNo + " is Odd";
      }
   }

   public static void main(String[] args) {
      System.out.println(markRollNumber(15)); // Odd
      System.out.println(markRollNumber(24)); // Even
   }
}
```

Reasoning to interviewer:

- I used int for roll numbers since they're always whole numbers.
- The % operator is efficient and avoids unnecessary loops or condition chains.
- This solution works for any range of roll numbers (small or large).

Extension (if interviewer digs deeper):

- If roll numbers were stored in a list, I could loop through and classify them:

```
int[] rollNumbers = {1, 2, 3, 4, 5};
for (int rollNo : rollNumbers) {
   System.out.println(markRollNumber(rollNo));
}
```

- In a real-world system, this could help seat students alternately in even/odd rows to avoid cheating.

Scenario 3: Gym Membership Classification

You're writing a Java feature for a gym membership system. The rules are:

- Less than 16 → Junior membership
- 16 to 59 → Standard membership
- 60 or more → Senior membership

Answer:

This is a straightforward case of decision-making using if-else statements. I'd use an int for age and simple range checks to assign the right membership.

Code Example:

```
public class GymMembership {
   public static String getMembershipType(int age) {
      if (age < 16) {
         return "Junior Membership";
      } else if (age >= 16 && age <= 59) {
         return "Standard Membership";
      } else {
         return "Senior Membership";
      }
   }

   public static void main(String[] args) {
      System.out.println(getMembershipType(12)); // Junior
      System.out.println(getMembershipType(30)); // Standard
      System.out.println(getMembershipType(65)); // Senior
   }
}
```

Reasoning to interviewer:

- I chose int for age because fractional ages don't make sense here.
- The conditions are mutually exclusive, so a clean if–else if–else chain works best.
- Placing < 16 first and >= 60 last ensures all ages are covered without overlap.

Real-world extension:

- If business rules change frequently (e.g., adding “Student” category), I could replace the if-else chain with a switch (Java 14+ switch expressions) or even store rules in a config file for flexibility.

Scenario 4: Food Ordering Program

You're creating a food-ordering program. The user types a number to select a meal:

- 1 → Pizza
- 2 → Burger
- 3 → Salad

Answer:
This is a case where a switch statement works better than multiple if-else blocks, because we're checking the same variable (mealChoice) against fixed values.

Code Example:

```
import java.util.Scanner;

public class FoodOrder {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.println("Enter your meal choice (1=Pizza, 2=Burger, 3=Salad):");
      int choice = sc.nextInt();

      String meal;
      switch (choice) {
         case 1:
            meal = "Pizza";
            break;
         case 2:
            meal = "Burger";
            break;
         case 3:
            meal = "Salad";
            break;
         default:
            meal = "Invalid choice";
      }

      System.out.println("You selected: " + meal);
   }
}
```

Reasoning to interviewer:

- A switch is cleaner than if-else when checking multiple fixed values for the same variable.
- I included a default case to handle invalid input, which is a best practice.
- Each case ends with break to prevent fall-through.

Java 14+ improvement (switch expressions):

```
String meal = switch (choice) {
   case 1 -> "Pizza";
   case 2 -> "Burger";
   case 3 -> "Salad";
   default -> "Invalid choice";
};
System.out.println("You selected: " + meal);
```

This newer syntax is more concise and avoids break statements.

Real-world extension:

- In a real food-ordering app, instead of just printing the meal, I'd probably map the choice to a database ID or menu object.

Scenario 5: Daily Savings Total

You're making a small finance app that needs to total the first 20 daily savings amounts:

- ₹1 on day 1,
- ₹2 on day 2,
- ...
- ₹20 on day 20

Answer:

This is a summation problem that can be solved with a simple for loop. I'll use an int for the daily values and total.

Code Example:

```
public class SavingsCalculator {
   public static void main(String[] args) {
      int total = 0;

      for (int day = 1; day <= 20; day++) {
         total += day;  // add daily saving to total
      }

      System.out.println("Total savings after 20 days: ₹" + total);
   }
}
```

Output:

Total savings after 20 days: ₹210

Reasoning to interviewer:

- I used a for loop because the number of days (20) is fixed.
- The accumulator variable total starts at 0 and keeps adding daily savings.

- Using int is fine here, but if the savings grew larger (like thousands per day), long would be safer.

Mathematical optimization:

Since this is the sum of the first *n* natural numbers, I could also use the formula:

```
Sum = n × (n + 1) / 2

For 20 days: 20 × 21 / 2 = 210.
```

Code (optimized):

```
int n = 20;
int total = n * (n + 1) / 2;
System.out.println("Total savings after 20 days: ₹" + total);
```

Real-world extension:

In a finance app, instead of fixed savings, amounts might come from user input or a file. Then I'd loop through that data source and keep an accumulator variable in the same way.

Scenario 6: Customer Feedback Validation

Imagine you're making a small customer feedback form in Java. Before you save the feedback, you must ensure the text isn't blank or missing, because empty data will make reports useless.

Answer:

This is a classic **String validation problem**. The two checks I need are:

1. The feedback should not be null.
2. It should not be empty or just spaces.

For this, I can use String methods like **isEmpty()** and **trim().**

Code Example:

```
public class FeedbackValidator {
   public static boolean isValidFeedback(String feedback) {
      return feedback != null && !feedback.trim().isEmpty();
   }

   public static void main(String[] args) {
      System.out.println(isValidFeedback("Great service!"));  // true
      System.out.println(isValidFeedback("   "));          // false
      System.out.println(isValidFeedback(null));           // false
   }
}
```

Reasoning to interviewer:

- I checked null first, otherwise calling methods like .trim() would cause a NullPointerException.
- I used trim() to remove extra spaces before checking emptiness, so " " won't be accepted.
- Returning a boolean makes it easy to integrate with a bigger system where feedback might be stored in a database only if validation passes.

Real-world extension:

- In a real feedback form, I might also limit maximum length (say 500 characters) to prevent abuse.
- If using Java 8+, I could also wrap this in an Optional<String> to handle the null case more elegantly.

Scenario 7: Restricted Product Batch

You're running a loop to list the products in stock, but as soon as you hit a certain product code (say, code 105), you want to stop listing further products because they are part of a restricted batch.

Answer:

This is exactly where the break statement helps. The idea is: keep looping through product codes, but stop completely once the restricted one is reached.

Code Example:

```
public class ProductListing {
   public static void main(String[] args) {
      int[] productCodes = {101, 102, 103, 104, 105, 106, 107};

      for (int code : productCodes) {
         if (code == 105) {
            System.out.println("Restricted batch found at code: " + code);
            break;  // stop further processing
         }
         System.out.println("Product in stock: " + code);
      }
   }
}
```

Output:

Product in stock: 101
Product in stock: 102
Product in stock: 103
Product in stock: 104
Restricted batch found at code: 105

Reasoning to interviewer:

- A for-each loop makes it clean to iterate over stock codes.
- I used break when I detected the restricted code, so no unnecessary processing happens after that.
- This is more efficient than looping through the entire array when you already know you need to stop early.

Real-world extension:

- Instead of hardcoding 105, the restricted codes could come from a database or a config file.
- If I only wanted to skip restricted items but keep listing others, I'd use continue instead of break.

Scenario 8: Skipping a Question Under Review

In an online quiz system, you want to list question numbers 1–10, but skip number 5 because it's under review.

Answer:

Here, instead of stopping the loop completely, I only want to skip one iteration when the number is 5. That's exactly what the continue statement does, it jumps to the next loop cycle.

Code Example:

```
public class QuizSystem {
   public static void main(String[] args) {
      for (int q = 1; q <= 10; q++) {
         if (q == 5) {
            System.out.println("Question " + q + " is under review. Skipping...");
            continue; // skip this one, go to next question
         }
         System.out.println("Displaying Question " + q);
      }
   }
}
```

Output:

```
Displaying Question 1
Displaying Question 2
Displaying Question 3
Displaying Question 4
Question 5 is under review. Skipping...
Displaying Question 6
Displaying Question 7
Displaying Question 8
Displaying Question 9
Displaying Question 10
```

Reasoning to interviewer:

- I used a for loop because the total number of questions is fixed (1 to 10).
- if (q == 5) detects the question under review.
- Instead of using break (which would stop the loop entirely), I used continue so that only question 5 is skipped and others still display.

Real-world extension:

- In a real quiz app, the “under review” status could come from a database instead of a hardcoded condition.
- If multiple questions needed skipping, I could store their IDs in a Set<Integer> and check membership inside the loop.

Scenario 9: Constant Role Codes

Your application assigns role codes like **ADMIN** and **CUSTOMER** that must never change after being set.

Answer:

In this case, I'd use the final keyword to make sure the role codes are constants. Once assigned, they can't be modified. To make them accessible across the application, I'd also declare them as public static.

Code Example:

```
public class Roles {
   public static final String ADMIN = "ADMIN";
   public static final String CUSTOMER = "CUSTOMER";
}

public class TestApp {
   public static void main(String[] args) {
      System.out.println("Role 1: " + Roles.ADMIN);
      System.out.println("Role 2: " + Roles.CUSTOMER);

      // Roles.ADMIN = "SUPERADMIN";  // ❌ Compile error: cannot assign a value to
final variable
   }
}
```

Reasoning to interviewer:

- I used public static final so that:
    - final → value can't be changed after initialization.
    - static → only one copy exists for the whole class.
    - public → accessible across the system without creating an object.
- This ensures role codes remain consistent everywhere, preventing accidental modifications.

Real-world extension:

- In larger applications, I might group constants in an enum instead of String constants:

```
public enum Role {
   ADMIN, CUSTOMER
}
```

Enums are safer because they restrict values to a fixed set and prevent typos like "ADMN".

Scenario 10: Temporary Discount Variable

You need a helper variable inside an if block to calculate a temporary discount, but it should not be accessible anywhere else.

Answer:

Here, the best approach is to use a local variable declared inside the if block. In Java, variables follow block scope, they only exist between the { } braces where they are declared.

Code Example:

```
public class DiscountCalculator {
   public static void main(String[] args) {
      int totalAmount = 1200;

      if (totalAmount > 1000) {
         double discount = totalAmount * 0.1; // local variable
         System.out.println("Discount applied: ₹" + discount);
         totalAmount -= discount;
      }

      // System.out.println(discount);  // Compile error: discount not visible here
      System.out.println("Final payable: ₹" + totalAmount);
   }
}
```

Reasoning to interviewer:

- I declared discount inside the if block because it's only needed there.
- This prevents misuse outside the block and makes code cleaner and more readable.
- Java automatically destroys local variables after the block finishes, which also saves memory.

Real-world extension:

- This principle of scope is important for things like temporary counters, loop variables, or validation flags.

- If a variable is needed across multiple methods, then I'd use instance variables instead, but for temporary calculations, local scope is the right choice.

Scenario 11: Number Classification

A number is entered by a user, and you need to tell whether it's positive, negative, or exactly zero.

Answer:

This is a straightforward use case for an if–else if–else chain. I'll store the user's input in an int and check its value.

Code Example:

```
import java.util.Scanner;

public class NumberCheck {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter a number: ");
      int num = sc.nextInt();

      if (num > 0) {
         System.out.println("The number is Positive");
      } else if (num < 0) {
         System.out.println("The number is Negative");
      } else {
         System.out.println("The number is Zero");
      }
   }
}
```

Reasoning to interviewer:

- I used an int because user inputs are typically whole numbers here, but if decimals were required (e.g., temperature readings), I could use double.
- I used a clean if–else if–else chain since the three conditions are mutually exclusive.
- The order matters: check positive first, then negative, then handle zero as the fallback case.

Real-world extension:

- This type of classification is common in reporting systems. For example, in a banking app:
    - Positive → deposit
    - Negative → withdrawal
    - Zero → no transaction

Scenario 12: Countdown Timer

You want to create a simple countdown timer that shows numbers from 5 down to 1.

Answer:

This is a perfect use case for a for loop running backwards**.** Instead of starting from 1 and going up, I'll start at 5 and decrement until 1.

Code Example:

```
public class Countdown {
   public static void main(String[] args) {
      for (int i = 5; i >= 1; i--) {
         System.out.println("Timer: " + i);
      }
      System.out.println("Time's up!");
   }
}
```

Output:

Timer: 5
Timer: 4
Timer: 3
Timer: 2
Timer: 1
Time's up!

Reasoning to interviewer:

- I chose a for loop because the countdown range is fixed.
- Initialization → i = 5, condition → i >= 1, update → i--.
- Using i-- makes the loop run backward.
- This shows how loop structure is flexible, it's not just for increasing sequences.

Real-world extension:

- A real countdown timer could include a Thread.sleep(1000) to pause for one second between prints.

```
for (int i = 5; i >= 1; i--) {
  System.out.println("Timer: " + i);
   try { Thread.sleep(1000); } catch (InterruptedException e) {}
}
```

- Similar logic is used in games, cooking apps, or online exam timers.

Scenario 13: Divisibility Check

In a math utility program, you must check whether a given number is divisible by both 4 and 6.

Answer:

This can be solved using the modulus (%) operator along with the logical AND (&&) operator. If a number leaves a remainder 0 when divided by both 4 and 6, then it qualifies.

Code Example:

```
import java.util.Scanner;

public class DivisibilityCheck {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter a number: ");
      int num = sc.nextInt();

      if (num % 4 == 0 && num % 6 == 0) {
         System.out.println(num + " is divisible by both 4 and 6");
      } else {
         System.out.println(num + " is NOT divisible by both 4 and 6");
      }
   }
}
```

Output Example:

- Input: 24 → divisible by both
- Input: 20 → not divisible by both

Reasoning to interviewer:

- % operator is the natural choice for divisibility checks.
- I used && because both conditions must be true at the same time.
- I chose an if-else since there are only two outcomes.

Real-world extension:

- This logic can extend to check divisibility by any set of numbers (e.g., scheduling tasks every 4 and 6 days).
- If divisibility by the least common multiple (LCM) matters, here it's 12, so num % 12 == 0 would be an optimized single check.

Scenario 14: Player Name Animation

A game stores the player's name in a string, and you want to display each character on a separate line for a cool animation effect.

Answer:

Since a String in Java can be treated like a sequence of characters, I can use the charAt() method inside a loop to print each character separately.

Code Example:

```
public class NameAnimation {
   public static void main(String[] args) {
      String playerName = "Rajat";

      for (int i = 0; i < playerName.length(); i++) {
         System.out.println(playerName.charAt(i));
      }
   }
}
```

Output:

R

A

J

a

t

Reasoning to interviewer:

- I used String.length() to decide loop boundaries.
- charAt(i) retrieves each character by index.
- A for loop is natural here since I know the exact number of iterations (length of the string).

Real-world extension:

- I could add a delay between characters for animation effect:

```
try { Thread.sleep(300); } catch (InterruptedException e) {}
```

- This technique is also used in typing effects on splash screens or loading animations.

Scenario 15: Month Selection for Hotel Booking

You're building a hotel booking application where the customer can select a travel month by entering a number between 1 and 12. Based on this number, the program should display the full month name. If the number is outside the range, display a clear error message.

Answer:

This is best solved with a switch statement, since we're mapping one number to one specific month.

Code Example (Classic Switch):

```
import java.util.Scanner;

public class BookingMonth {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter month number (1-12): ");
      int month = sc.nextInt();

      String monthName;
      switch (month) {
         case 1:  monthName = "January"; break;
         case 2:  monthName = "February"; break;
         case 3:  monthName = "March"; break;
         case 4:  monthName = "April"; break;
         case 5:  monthName = "May"; break;
         case 6:  monthName = "June"; break;
         case 7:  monthName = "July"; break;
         case 8:  monthName = "August"; break;
         case 9:  monthName = "September"; break;
         case 10: monthName = "October"; break;
         case 11: monthName = "November"; break;
         case 12: monthName = "December"; break;
         default: monthName = "Invalid month. Please enter a number between 1 and
12.";
      }

      System.out.println(monthName);
   }
}
```

Output Example:

- Input: 5 → May
- Input: 15 → Invalid month. Please enter a number between 1 and 12.

Reasoning to interviewer:

- switch is better than 12 chained if-else conditions because the variable being checked is the same.
- Including a default case ensures invalid input is handled gracefully.
- Using a String for month name makes the code clean and readable.

Java 14+ Modern Switch Expression:

```
String monthName = switch (month) {
   case 1  -> "January";
   case 2  -> "February";
   case 3  -> "March";
   case 4  -> "April";
   case 5  -> "May";
   case 6  -> "June";
   case 7  -> "July";
   case 8  -> "August";
   case 9  -> "September";
   case 10 -> "October";
   case 11 -> "November";
   case 12 -> "December";
   default -> "Invalid month. Please enter a number between 1 and 12.";
};
```

This avoids break statements and is less error-prone.

Real-world extension:

- In a real hotel booking system, instead of just printing the month, I could use this value to filter available seasonal packages.

Scenario 16: Password Length Check for Login

You are creating a login system for an e-commerce site. One of the rules is that the user's password must be at least 8 characters long for security reasons. The system should give feedback if the password is too short.

Answer:

Here I'd use the String.length() method to check the number of characters in the password. If it's less than 8, I'll reject it with a clear message.

Code Example:

```
import java.util.Scanner;

public class LoginSystem {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter your password: ");
      String password = sc.nextLine();

      if (password == null || password.length() < 8) {
         System.out.println("Password too short. Must be at least 8 characters.");
      } else {
         System.out.println("Password accepted. Proceeding to login...");
      }
   }
}
```

Output Example:

- Input: pass12 → Password too short.
- Input: mypassword → Password accepted.

Reasoning to interviewer:

- I used String.length() to check size, which counts all characters including spaces and symbols.
- A null check ensures we don't hit a NullPointerException.
- This simple check enforces a basic security rule without involving regex or advanced logic.

Real-world extension:

- In real systems, we'd add more rules like requiring uppercase letters, numbers, and special characters, possibly using regex.

- For sensitive data like passwords, I'd also use char[] instead of String in secure systems, because String values stay in memory longer due to immutability.

Scenario 17: Factorial Calculation for Statistics Module

Your program needs to find the factorial of a number for a mathematical calculation module in a statistics application.

Answer:

Factorial means multiplying all numbers from 1 up to n. For example, 5! = 5 × 4 × 3 × 2 × 1 = 120.
This can be done using a simple loop.

Code Example (Using for loop):

```
import java.util.Scanner;

public class FactorialCalculator {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter a number: ");
      int n = sc.nextInt();

      long factorial = 1; // long to handle large results
      for (int i = 1; i <= n; i++) {
         factorial *= i;
      }

      System.out.println("Factorial of " + n + " is: " + factorial);
   }
}
```

Example Output:

- Input: 5 → Factorial of 5 is: 120
- Input: 0 → Factorial of 0 is: 1

Reasoning to interviewer:

- I used a for loop because the number of iterations is fixed (from 1 to n).
- Used long instead of int because factorial grows very fast (e.g., 20! exceeds int).
- Handled the base case: 0! = 1.

Alternative (Using recursion):

```
public static long factorial(int n) {
   if (n == 0 || n == 1) return 1;
```

```
    return n * factorial(n - 1);
}
```

Real-world extension:

- Factorial calculations are often used in statistics (e.g., permutations and combinations).
- For very large numbers (like 100!), BigInteger is needed because even long will overflow.

Scenario 18: Summing Even Numbers for Financial Analysis

In a financial analysis program, you want to sum all even numbers from 2 to 20 for a specific dataset check.

Answer:

This can be solved by running a loop and checking if each number is even using the modulus operator. Alternatively, I can increment by 2 to directly cover even numbers.

Code Example (Checking with %):

```
public class EvenSum {
   public static void main(String[] args) {
      int sum = 0;
      for (int i = 2; i <= 20; i++) {
         if (i % 2 == 0) {
            sum += i;
         }
      }
      System.out.println("Sum of even numbers from 2 to 20: " + sum);
   }
}
```

Output:

Sum of even numbers from 2 to 20: 110

Optimized Code (Increment by 2):

```
int sum = 0;
for (int i = 2; i <= 20; i += 2) {
   sum += i;
}
System.out.println("Sum of even numbers from 2 to 20: " + sum);
```

Reasoning to interviewer:

- Using i += 2 is more efficient since we skip odd numbers entirely.
- Used an accumulator variable sum that keeps adding values across iterations.
- The problem size is small, but this approach scales well if the range is larger.

Mathematical Optimization:

This is an arithmetic series: 2 + 4 + ... + 20.
Formula:

Sum = n/2 × (first + last)

Here, n = 10 terms → 10/2 × (2 + 20) = 5 × 22 = 110.

Real-world extension:

- In financial datasets, such checks can be used for data validation (e.g., ensuring even-day transactions are being summed separately).

Scenario 19: Square of a Number

In a geometry calculator app, the user enters a number and the program should display its square (number × number).

Answer:

This can be solved directly using the multiplication operator. I'll take input from the user, store it in an integer (or double if decimals are expected), and then multiply the number by itself.

Code Example (Integer):

```
import java.util.Scanner;

public class SquareCalculator {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter a number: ");
      int num = sc.nextInt();

      int square = num * num;
      System.out.println("Square of " + num + " is: " + square);
   }
}
```

Example Output:

- Input: 5 → Output: Square of 5 is: 25
- Input: 12 → Output: Square of 12 is: 144

Code Example (Double for decimals):

```
double num = sc.nextDouble();
double square = num * num;
```

Reasoning to interviewer:

- Used int for whole numbers, but double if fractional inputs like 2.5 are expected.
- The multiplication operator is the simplest approach, no need for libraries.

- Could also use Math.pow(num, 2), but multiplication is faster and more readable here.

Real-world extension:

- In a geometry app, this could be used for calculating area of a square (side × side).
- For large calculations (like big exponents), Math.pow() or even BigInteger might be used.

Scenario 20: Event Countdown Timer

In an event management app, you need a countdown timer that starts from a given number and shows each second until zero.

Answer:

Here, a while loop is a natural fit since I don't know in advance what number the countdown might start from, it could be 5, 10, or 60 seconds. The loop keeps running until it reaches zero.

Code Example:

```
import java.util.Scanner;

public class EventCountdown {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter countdown start: ");
      int seconds = sc.nextInt();

      while (seconds > 0) {
         System.out.println("Time left: " + seconds + "s");
         try {
            Thread.sleep(1000); // pause 1 second
         } catch (InterruptedException e) {
            e.printStackTrace();
         }
         seconds--; // decrement
      }

      System.out.println("Event Started!");
   }
}
```

Example Output (if input = 5):

Time left: 5s
Time left: 4s
Time left: 3s
Time left: 2s
Time left: 1s
Event Started!

Reasoning to interviewer:

- A while loop works best since the number of iterations depends on user input.
- Added Thread.sleep(1000) to simulate a real countdown timer (1 second per step).
- Loop condition seconds > 0 ensures it stops exactly at zero.

Real-world extension:

- Could be used for exam countdowns, flash sales, or live event launches.
- If this needed to run in the background without blocking other tasks, I'd use a separate thread or a scheduled executor service.

Scenario 21: Order Status Handling in E-commerce

You are working on an e-commerce platform where each order has a status such as "PLACED", "SHIPPED", or "DELIVERED".
Depending on the status, the program should trigger different actions, such as sending notifications or updating the inventory.

Answer:

Since order status has fixed possible values, this is a good fit for either a switch on String (Java 7+) or better, an enum.

Code Example (Using switch with String):

```
public class OrderProcessor {
   public static void main(String[] args) {
      String status = "SHIPPED";  // example order status

      switch (status) {
         case "PLACED":
            System.out.println("Send order confirmation email");
            break;
         case "SHIPPED":
            System.out.println("Send shipping notification and update tracking");
            break;
         case "DELIVERED":
            System.out.println("Send delivery confirmation and update inventory");
            break;
         default:
            System.out.println("Unknown status");
      }
   }
}
```

Output (if status = "SHIPPED"):

Send shipping notification and update tracking

Reasoning to interviewer:

- I used a switch because the status can only be one of a small set of known values.
- Using a default case ensures unrecognized statuses don't break the flow.
- If statuses grow beyond 3–4, an enum is safer because it prevents typos and restricts values.

Better Approach (Using Enums):

```
enum OrderStatus {
   PLACED, SHIPPED, DELIVERED
}

public class OrderProcessor {
   public static void main(String[] args) {
      OrderStatus status = OrderStatus.SHIPPED;

      switch (status) {
         case PLACED -> System.out.println("Send order confirmation email");
         case SHIPPED -> System.out.println("Send shipping notification and update
tracking");
         case DELIVERED -> System.out.println("Send delivery confirmation and update
inventory");
      }
   }
}
```

Why enums are better:

- Prevents mistakes like "SHIPPPED" typo (which a String would allow).
- Provides type safety and better readability.

Real-world extension:

- In an actual e-commerce system, instead of just printing actions, each case would trigger service calls like:

- NotificationService.sendEmail()
- InventoryService.updateStock()
- TrackingService.updateShipment()

Scenario 22: Reversing a Sentence in a Text Editor

You are creating a text editor feature where the user can reverse a sentence they type. For example, typing "Hello World" should output "dlroW olleH".

Answer:

Reversing a string can be done in multiple ways: using a loop, using StringBuilder, or using StringBuffer. Since strings are immutable in Java, I'll prefer StringBuilder for efficiency.

Code Example (Using StringBuilder):

```
import java.util.Scanner;

public class ReverseSentence {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter a sentence: ");
      String input = sc.nextLine();

      String reversed = new StringBuilder(input).reverse().toString();
      System.out.println("Reversed sentence: " + reversed);
   }
}
```

Example Output:

- Input: Hello World
- Output: dlroW olleH

Reasoning to interviewer:

- I used StringBuilder because it provides a built-in reverse() method and is more efficient than manual concatenation.
- Strings in Java are immutable, so directly reversing them with concatenation would create multiple new objects, which is inefficient.
- If the program needs to be thread-safe, I could use StringBuffer instead of StringBuilder.

Alternative (Using a loop manually):

```
String sentence = "Hello World";
String reversed = "";
for (int i = sentence.length() - 1; i >= 0; i--) {
   reversed += sentence.charAt(i); // not efficient for large strings
}
System.out.println(reversed);
```

Real-world extension:

- This feature could be extended to reverse each word individually while keeping word order intact (e.g., "Hello World" → "olleH dlroW").
- Such string manipulation is often used in text editors, search engines, and word processors.

Scenario 23: Highest and Lowest Temperatures

In a weather data analysis program, you need to find the highest and lowest recorded temperatures from an array of daily readings.

Answer:

To solve this, I'll iterate over the array once and keep track of two variables: one for the maximum temperature and one for the minimum temperature.

Code Example:

```
public class TemperatureAnalysis {
   public static void main(String[] args) {
      int[] temps = {32, 35, 30, 40, 28, 36, 33};

      int max = temps[0]; // assume first element is max
      int min = temps[0]; // assume first element is min

      for (int t : temps) {
         if (t > max) max = t;
         if (t < min) min = t;
      }

      System.out.println("Highest temperature: " + max);
      System.out.println("Lowest temperature: " + min);
   }
}
```

Output (for sample data):

Highest temperature: 40

Lowest temperature: 28

Reasoning to interviewer:

- I initialized max and min with the first element so that the loop can compare from the start.

- Each iteration updates max and min only when necessary.
- This is a linear time algorithm (O(n)), efficient even for large datasets.

Real-world extension:

- If the dataset comes from a file or API, I'd first load it into an array or List<Integer>.
- For large-scale analytics, I could use Java Streams:

```
int max = Arrays.stream(temps).max().getAsInt();
int min = Arrays.stream(temps).min().getAsInt();
```

- In real weather apps, additional logic might include handling missing values or abnormal spikes (like sensor errors).

Scenario 24: Email Validation in Signup Form

In a signup form for a job portal, you need to verify that a provided email is valid by checking for an "@" symbol and a common domain ending like ".org".

Answer:

A simple way to validate an email is by checking if it contains "@" and ends with the correct domain using built-in String methods. For full-scale validation, regex is used, but for a basics-level check, contains() and endsWith() are enough.

Code Example:

```
import java.util.Scanner;

public class EmailValidator {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter your email: ");
      String email = sc.nextLine();

      if (email != null && email.contains("@") && email.endsWith(".org")) {
         System.out.println("Valid email address.");
      } else {
         System.out.println("Invalid email address. Please provide a valid .org email.");
      }
   }
}
```

Example Output:

- Input: user@example.org → Valid email address.
- Input: userexample.com → Invalid email address.

Reasoning to interviewer:

- Used contains("@") to check the structure has the basic user@domain format.
- Used endsWith(".org") to ensure it matches the required domain type.
- Added a null check to prevent accidental NullPointerException.

Real-world extension:

- For production systems, I'd use regex for stricter checks:

  email.matches("^[A-Za-z0-9+_.-]+@(.+)$")

- And often use libraries like Jakarta Validation (@Email**)** when working with frameworks (e.g., Spring Boot).

Scenario 25: Exponential Calculation in Calculator App

You are building a calculator app that needs to compute exponential values, such as calculating 5^3.

Answer:

In Java, there's no direct ^ operator for exponentiation (the ^ symbol is actually bitwise XOR). The correct way is to use the Math.pow() method or implement it manually with a loop.

Code Example (Using Math.pow):

```
public class ExponentialCalculator {
   public static void main(String[] args) {
      int base = 5;
      int exponent = 3;

      double result = Math.pow(base, exponent);
      System.out.println(base + "^" + exponent + " = " + (int) result);
   }
}
```

Output:

5^3 = 125

Alternative (Using a loop):

```
int base = 5, exponent = 3;
int result = 1;

for (int i = 1; i <= exponent; i++) {
   result *= base;
}
System.out.println(base + "^" + exponent + " = " + result);
```

Reasoning to interviewer:

- Math.pow() is concise and built-in, but it returns a double, so I cast to int when I know the result is whole.
- A manual loop works if I want to keep everything as integers.
- For negative exponents, Math.pow() handles fractional results correctly (e.g., 5^-2 = 0.04).

Real-world extension:

- Used in calculators, finance apps (compound interest), and scientific applications.
- For very large powers (like 1000^1000), I'd use BigInteger to avoid overflow.

Scenario 26: Safe String-to-Integer Conversion

You are building a Java tool that reads a numeric string from a file, such as "4567", and you need to convert it into an actual integer so it can be used in calculations. However, you also want to make sure the program doesn't crash if the file contains something invalid, like "abc".

Answer:

To convert a string into an integer, I can use Integer.parseInt(). But since it throws a NumberFormatException if the string is not a valid number, I'll handle it with a try-catch block.

Code Example:

```
public class StringToIntConverter {
   public static void main(String[] args) {
      String input = "4567";   // assume this was read from a file

      try {
         int number = Integer.parseInt(input);
         System.out.println("Converted number: " + number);
         System.out.println("Number + 10 = " + (number + 10));
      } catch (NumberFormatException e) {
         System.out.println("Invalid number format: " + input);
      }
   }
}
```

Example Outputs:

- Input: "4567" → Converted number: 4567
- Input: "abc" → Invalid number format: abc

Reasoning to interviewer:

- Integer.parseInt() is the standard way to convert strings to integers.
- Wrapped it in try-catch so that invalid inputs don't crash the program.
- If the file might contain very large numbers (beyond int), I'd use Long.parseLong() or even BigInteger.

Real-world extension:

- This type of conversion is common in data ingestion tools (reading numeric values from CSV/JSON/XML).
- Instead of rejecting invalid values outright, I could log them for review or store them in a separate “invalid data” file.

Scenario 27: Summing Sales Data in a 2D Array

Your Java analytics program stores monthly sales data in a two-dimensional array — rows for each year, and columns for months. You need to calculate the total sales over all years.

Answer:

To handle a 2D array, I’ll use a nested loop**:**

- Outer loop → iterate over each year (row).
- Inner loop → iterate over each month (column).
- Keep an accumulator variable for the total.

Code Example:

```
public class SalesAnalysis {
   public static void main(String[] args) {
      // Rows = years, Columns = months
      int[][] sales = {
         {1200, 1500, 1800, 2000},   // Year 1
         {1400, 1600, 1700, 2100},   // Year 2
         {1300, 1550, 1900, 2200}    // Year 3
      };

      int totalSales = 0;

      for (int year = 0; year < sales.length; year++) {
         for (int month = 0; month < sales[year].length; month++) {
            totalSales += sales[year][month];
         }
      }

      System.out.println("Total Sales across all years: " + totalSales);
   }
}
```

Output (for given sample):

Total Sales across all years: 19350

Reasoning to interviewer:

- Used a nested for loop because I need to traverse both dimensions.
- Chose sales.length for years and sales[year].length for months — this makes it flexible if some years have fewer/more months recorded.
- Maintained a single accumulator variable to sum up all values.

Real-world extension:

- In real analytics apps, this logic could be adapted to compute yearly totals, monthly averages, or highest sales month.
- With Java 8+, I could also flatten the 2D array using streams:

```
int total = Arrays.stream(sales)
        .flatMapToInt(Arrays::stream)
        .sum();
```

Scenario 28: Fibonacci Sequence for a Math Learning App

In a Java math learning app, you want to display the Fibonacci sequence up to a certain number of terms for practice problems. The sequence starts at 0 and 1, with each new number being the sum of the two before it.

Answer:

The Fibonacci sequence is: 0, 1, 1, 2, 3, 5, 8.... Each number is the sum of the previous two. I can generate it with a loop by maintaining two variables for the last two numbers.

Code Example:

```
import java.util.Scanner;

public class FibonacciSequence {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter number of terms: ");
      int n = sc.nextInt();

      int a = 0, b = 1;

      System.out.print("Fibonacci sequence: ");
      for (int i = 1; i <= n; i++) {
         System.out.print(a + " ");
         int next = a + b;
         a = b;
         b = next;
      }
   }
}
```

Example Output (for n = 7):

Fibonacci sequence: 0 1 1 2 3 5 8

Reasoning to interviewer:

- I started with a=0, b=1, then generated each new number as next = a + b.
- After printing each term, I updated a and b for the next iteration.
- Used a for loop since the number of terms (n) is known.

Alternative (Recursion):

```
public static int fib(int n) {
   if (n <= 1) return n;
   return fib(n - 1) + fib(n - 2);
}
```

But recursion is less efficient due to repeated calculations — loops are better for practice apps.

Real-world extension:

- This kind of sequence generation is useful for practice problems, quizzes, or number pattern games in math learning apps.
- For very large n, I'd use BigInteger instead of int to avoid overflow.

Scenario 29: Palindrome Check for Word Puzzle Game

You're building a word puzzle game in Java, and you need to check if the user's input word is a palindrome (reads the same forward and backward), such as "level" or "racecar".

Answer:

A palindrome check means comparing a word to its reverse. If both are the same, the word is a palindrome.

Code Example (Using StringBuilder):

```
import java.util.Scanner;

public class PalindromeCheck {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter a word: ");
      String word = sc.nextLine();

      String reversed = new StringBuilder(word).reverse().toString();

      if (word.equals(reversed)) {
         System.out.println(word + " is a palindrome.");
      } else {
         System.out.println(word + " is not a palindrome.");
      }
   }
}
```

Example Output:

- Input: level → level is a palindrome.
- Input: java → java is not a palindrome.

Reasoning to interviewer:

- I used StringBuilder.reverse() because it's efficient and avoids writing manual reversal logic.
- equals() is necessary instead of ==, because I want content comparison of the two strings.
- Works for any input, short or long.

Alternative (Using loop manually):

```
boolean isPalindrome = true;
for (int i = 0; i < word.length() / 2; i++) {
   if (word.charAt(i) != word.charAt(word.length() - i - 1)) {
      isPalindrome = false;
      break;
   }
}
```

This avoids creating a reversed string, and directly compares characters.

Real-world extension:

- Useful in word games, puzzle apps, or even in interview puzzles.
- With slight modification, it can be extended to check numeric palindromes (like 121, 1331).

Scenario 30: Basic Login Simulation

You're creating a very basic login simulation for a Java course assignment. The program should check if the username and password entered by the user match predefined credentials and display an appropriate message.

Answer:

In Java, == compares object references, not string content. So to check whether two strings have the same content, we must use .equals().

Code Example:

```
import java.util.Scanner;

public class LoginSimulation {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      String correctUsername = "admin";
      String correctPassword = "password123";

      System.out.print("Enter username: ");
      String username = sc.nextLine();

      System.out.print("Enter password: ");
      String password = sc.nextLine();

      if (username.equals(correctUsername) && password.equals(correctPassword)) {
         System.out.println("Login successful! Welcome " + username);
      } else {
         System.out.println("Invalid username or password.");
      }
   }
}
```

Example Output:

- Input: admin / password123 → Login successful! Welcome admin
- Input: admin / pass123 → Invalid username or password.

Reasoning to interviewer:

- I used .equals() instead of == because string values must be compared by content, not memory reference.
- Both username and password checks are combined with && (logical AND).
- In real-world systems, passwords should never be stored in plain text — they should be hashed and checked securely.

Common Mistake (if using ==):

if (username == correctUsername) { … } // may fail even if values look the same

This fails because == only checks if both variables point to the same memory location.

Real-world extension:

- Instead of hardcoded credentials, I'd pull them from a database.
- I'd also use char[] for password handling in secure apps, since String is immutable and lingers in memory.

Scenario 31: Role-Based Access Control

Your Java app manages users with roles like "Admin" and "User." Different roles should have different access rights.

Answer:

The simplest way is to check the role string and trigger actions based on it. For safety and readability, I'd prefer equals() for string comparison, or better, an enum for roles.

Code Example (Using if-else with Strings):

```
public class RoleAccess {
   public static void main(String[] args) {
      String role = "Admin";  // example input

      if (role.equals("Admin")) {
         System.out.println("Access granted: Full control.");
      } else if (role.equals("User")) {
         System.out.println("Access granted: Limited features.");
      } else {
         System.out.println("Access denied: Unknown role.");
      }
   }
}
```

Output:

- Role = Admin → Full control
- Role = User → Limited features
- Role = Guest → Access denied

Reasoning to interviewer:

- Used .equals() for content comparison (not ==).
- This structure ensures each role gets specific rights, and unrecognized roles are blocked.
- But relying on raw strings is error-prone (e.g., "admin" vs "Admin" typo).

Better Approach (Using Enums):

```
enum Role { ADMIN, USER }

public class RoleAccess {
   public static void main(String[] args) {
      Role role = Role.ADMIN;

      switch (role) {
         case ADMIN -> System.out.println("Access granted: Full control.");
         case USER  -> System.out.println("Access granted: Limited features.");
      }
   }
}
```

Why enums are better:

- Prevent typos (can't accidentally write "Admni").
- Provide type safety and better readability.
- Easier to extend later if more roles are added (e.g., GUEST, MODERATOR).

Real-world extension:

- In real applications, roles are often stored in databases and checked during authentication.
- Frameworks like Spring Security manage this with annotations (@PreAuthorize("hasRole('ADMIN')")).

Scenario 32: Finding GCD of Two Numbers

You want to find the Greatest Common Divisor (GCD) of two integers input by users.

Answer:

The GCD of two numbers is the largest number that divides both without leaving a remainder. A simple way is to use the Euclidean Algorithm.

Code Example (Euclidean Algorithm with loop):

```
import java.util.Scanner;

public class GCDCalculator {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter first number: ");
      int a = sc.nextInt();
      System.out.print("Enter second number: ");
      int b = sc.nextInt();

      while (b != 0) {
         int temp = b;
         b = a % b;
         a = temp;
      }

      System.out.println("GCD is: " + a);
   }
}
```

Example Output:

- Input: 54 and 24 → GCD is: 6
- Input: 20 and 8 → GCD is: 4

Reasoning to interviewer:

- I chose the Euclidean Algorithm because it's efficient (O(log(min(a,b)))).
- The loop continues until the remainder becomes 0.
- The final value of a is the GCD.

Alternative (Naive approach):

```
int gcd = 1;
for (int i = 1; i <= a && i <= b; i++) {
   if (a % i == 0 && b % i == 0) {
      gcd = i;
   }
}
```

This works but is slower (O(min(a, b))).

Real-world extension:

- GCD is used in cryptography (RSA key generation), reducing fractions, and scheduling tasks that must align on common intervals.

Scenario 33: Character Frequency in a String

You want to count how often each character appears in a given string.

Answer:

I'll use a HashMap where each character is the key and its count is the value. I'll loop through the string and update the map.

Code Example:

```
import java.util.HashMap;
import java.util.Map;

public class CharFrequency {
   public static void main(String[] args) {
      String input = "programming";
      Map<Character, Integer> freq = new HashMap<>();

      for (char c : input.toCharArray()) {
         freq.put(c, freq.getOrDefault(c, 0) + 1);
      }

      System.out.println("Character frequencies: " + freq);
   }
}
```

Example Output:

Character frequencies: {p=1, r=2, o=1, g=2, a=1, m=2, i=1, n=1}

Reasoning to interviewer:

- Used toCharArray() to loop over characters easily.
- HashMap stores counts efficiently.
- getOrDefault() avoids writing extra if-else logic for first-time characters.

Alternative (without Map, using array):

If input is only lowercase alphabets:

```
int[] count = new int[26];
for (char c : input.toCharArray()) {
   count[c - 'a']++;
}
```

This is more space-efficient but limited to known ranges.

Real-world extension:

- Character frequency is used in text analysis**,** encryption algorithms, and spell-checkers**.**
- Extended version could count words instead of characters.

Scenario 34: Prime Number Check

You want to check if a given integer is prime (only divisible by 1 and itself).

Answer:

A prime number is greater than 1 and divisible only by 1 and itself. To check primality, I can test divisibility up to the square root of the number.

Code Example:

```
import java.util.Scanner;

public class PrimeCheck {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter a number: ");
      int num = sc.nextInt();

      boolean isPrime = true;

      if (num <= 1) {
         isPrime = false; // 0 and 1 are not prime
      } else {
         for (int i = 2; i <= Math.sqrt(num); i++) {
            if (num % i == 0) {
               isPrime = false;
               break;
            }
         }
      }

      if (isPrime) {
         System.out.println(num + " is a prime number.");
      } else {
         System.out.println(num + " is not a prime number.");
      }
   }
}
```

Example Output:

- Input: 7 → 7 is a prime number.
- Input: 12 → 12 is not a prime number.

Reasoning to interviewer:

- Checked numbers up to √num instead of num-1, making it much faster for large numbers.
- Handled edge cases (<= 1 not prime).
- Used a boolean flag isPrime to store the result.

Alternative (Quick one-liner with Streams, Java 8+):

```
boolean isPrime = num > 1 && IntStream.rangeClosed(2, (int)Math.sqrt(num))
                             .allMatch(i -> num % i != 0);
```

Real-world extension:

- Prime number checks are often used in encryption (RSA keys), random number generation, and hashing algorithms.
- For very large numbers, probabilistic tests (like Miller-Rabin) are used instead of simple loops.

Scenario 35: Sorting an Integer Array

You need to sort an integer array in ascending order for data processing.

Answer:

The simplest way is to use Java's built-in sorting methods, but I can also implement a basic sorting algorithm to show understanding.

Code Example (Using Arrays.sort):

```
import java.util.Arrays;

public class SortArray {
   public static void main(String[] args) {
      int[] numbers = {5, 2, 9, 1, 7};

      Arrays.sort(numbers); // sorts in ascending order

      System.out.println("Sorted array: " + Arrays.toString(numbers));
   }
}
```

Output:

Sorted array: [1, 2, 5, 7, 9]

Reasoning to interviewer:

- Arrays.sort() uses Dual-Pivot Quicksort for primitives (very efficient).
- The method sorts the array in-place, so no extra memory is needed.
- Time complexity is O(n log n).

Alternative (Manual Bubble Sort for basics):

```
int[] numbers = {5, 2, 9, 1, 7};

for (int i = 0; i < numbers.length - 1; i++) {
   for (int j = 0; j < numbers.length - i - 1; j++) {
      if (numbers[j] > numbers[j + 1]) {
         int temp = numbers[j];
         numbers[j] = numbers[j + 1];
         numbers[j + 1] = temp;
      }
```

```
   }
}
System.out.println("Sorted array: " + Arrays.toString(numbers));
```

Real-world extension:

- Sorting is used in data processing, analytics, ranking, and search optimizations**.**
- For large datasets, I'd use parallel sort**:**

```
Arrays.parallelSort(numbers);
```

which leverages multithreading for performance.

Scenario 36: Factorial of Large Numbers

You want to compute the factorial of large numbers that exceed the limits of primitive data types.

Answer:

Since factorial grows extremely fast, even long (which can hold up to ~9e18) overflows quickly. For larger factorials (like 50!, 100!), I'll use BigInteger from java.math because it can handle arbitrarily large numbers.

Code Example (Using BigInteger):

```
import java.math.BigInteger;
import java.util.Scanner;

public class BigFactorial {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter a number: ");
      int n = sc.nextInt();

      BigInteger factorial = BigInteger.ONE;

      for (int i = 1; i <= n; i++) {
         factorial = factorial.multiply(BigInteger.valueOf(i));
      }

      System.out.println(n + "! = " + factorial);
   }
}
```

Example Output (n = 20):

20! = 2432902008176640000

For n = 100, the number is 158 digits long, which would overflow long.

Reasoning to interviewer:

- I used BigInteger.ONE as the starting value.
- Each loop multiplies the running factorial by the next integer, converted via BigInteger.valueOf(i).

- BigInteger supports operations like multiply, add, subtract, etc., instead of operators (*, +) because primitives can't handle unlimited size.

Real-world extension:

- Large factorials are used in combinatorics, probability, cryptography, and scientific simulations.
- For extremely huge numbers, even BigInteger computations get slow, so optimizations or approximate methods (like Stirling's formula) may be used.

Scenario 37: Removing Duplicates from an Integer Array

You have an integer array with duplicates and want to get an array without repeated elements.

Answer:

The simplest way in Java is to use a Set, because a Set automatically eliminates duplicates. After that, if needed, I can convert it back to an array.

Code Example (Using HashSet):

```
import java.util.*;

public class RemoveDuplicates {
   public static void main(String[] args) {
      int[] numbers = {1, 2, 3, 2, 4, 1, 5};

      // Convert array to Set
      Set<Integer> uniqueSet = new HashSet<>();
      for (int num : numbers) {
         uniqueSet.add(num);
      }

      // Convert Set back to array
      int[] uniqueArray = uniqueSet.stream().mapToInt(Integer::intValue).toArray();

      System.out.println("Unique elements: " + Arrays.toString(uniqueArray));
   }
}
```

Example Output:

Unique elements: [1, 2, 3, 4, 5]

Reasoning to interviewer:

- HashSet automatically removes duplicates.
- Used stream().mapToInt() to convert the set back into an int[].
- Order is not guaranteed with HashSet. If order matters, I'd use a LinkedHashSet (preserves insertion order) or TreeSet (sorts elements).

Alternative (Manual without Set):

- Sort the array and then loop through, copying only unique elements into a new array. This works but requires more code and sorting overhead.

Real-world extension:

- Removing duplicates is often required in data cleaning, analytics, and user record management.
- In big systems, deduplication is also applied on files, logs, or database records.

Scenario 38: Formatting a List of Integers into a Comma-Separated String

You want to transform a list of integers into a nicely formatted string with commas separating the numbers.

Answer:

In Java, the easiest way is to use the String.join method (after converting integers to strings) or Java Streams with Collectors.joining().

Code Example (Using Streams):

```
import java.util.*;
import java.util.stream.Collectors;

public class ListFormatter {
   public static void main(String[] args) {
      List<Integer> numbers = Arrays.asList(10, 20, 30, 40, 50);

      String formatted = numbers.stream()
                        .map(String::valueOf)   // convert each number to String
                        .collect(Collectors.joining(", "));

      System.out.println("Formatted String: " + formatted);
   }
}
```

Output:

Formatted String: 10, 20, 30, 40, 50

Reasoning to interviewer:

- Used .map(String::valueOf) to convert each integer into a string.
- Used Collectors.joining(", ") to add commas with spaces in between.
- This approach is clean and works for any list size.

Alternative (Manual with StringBuilder):

```
List<Integer> numbers = Arrays.asList(10, 20, 30, 40, 50);
StringBuilder sb = new StringBuilder();

for (int i = 0; i < numbers.size(); i++) {
   sb.append(numbers.get(i));
   if (i < numbers.size() - 1) {
      sb.append(", ");
   }
}
System.out.println("Formatted String: " + sb.toString());
```

Real-world extension:

- This is commonly used in report generation, logs, exporting CSV files, and displaying results in dashboards.
- With slight modification, I could change the delimiter (e.g., ; or |) or even wrap values in quotes for CSV export.

Scenario 39: Matrix Exponentiation

You want to raise a square matrix to a given power (matrix exponentiation).

Answer:

Matrix exponentiation means multiplying a square matrix by itself repeatedly. For example, $A^2 = A \times A$, $A^3 = A \times A \times A$.

To do this:

1. Write a method for matrix multiplication.
2. Use that method in a loop to multiply the matrix by itself the required number of times.

Code Example (Naive Implementation):

```
import java.util.Arrays;

public class MatrixExponentiation {
  // Method for multiplying two matrices
  public static int[][] multiply(int[][] A, int[][] B) {
    int n = A.length;
    int[][] result = new int[n][n];

    for (int i = 0; i < n; i++) {
      for (int j = 0; j < n; j++) {
        for (int k = 0; k < n; k++) {
          result[i][j] += A[i][k] * B[k][j];
        }
      }
    }
    return result;
  }

  // Raise matrix to a given power
  public static int[][] power(int[][] A, int exp) {
    int n = A.length;
    int[][] result = new int[n][n];

    // Initialize result as identity matrix
    for (int i = 0; i < n; i++) {
      result[i][i] = 1;
    }

    // Multiply matrix exp times
    for (int i = 0; i < exp; i++) {
```

```
            result = multiply(result, A);
        }

        return result;
    }

    public static void main(String[] args) {
        int[][] matrix = {
            {1, 2},
            {3, 4}
        };

        int exp = 2; // Example: raise to power 2
        int[][] result = power(matrix, exp);

        System.out.println("Matrix raised to power " + exp + ":");
        for (int[] row : result) {
            System.out.println(Arrays.toString(row));
        }
    }
}
```

Output (Matrix^2):

[7, 10]
[15, 22]

Reasoning to interviewer:

- I initialized the result as the identity matrix, because multiplying by the identity matrix leaves the original matrix unchanged.
- Used nested loops for matrix multiplication: $O(n^3)$ time complexity.
- The naive power method multiplies the matrix exp times, so complexity is $O(exp \times n^3)$.

Optimized Approach (Exponentiation by Squaring):
Instead of multiplying exp times, I could reduce complexity with **divide and conquer**:

- If exp is even → A^exp = (A^(exp/2)) × (A^(exp/2)).
- If odd → A^exp = A × (A^(exp-1)).

This brings the complexity down to $O(\log exp \times n^3)$.

Real-world extension:

- Matrix exponentiation is heavily used **in** dynamic programming, graph algorithms, cryptography, and linear recurrence relations like Fibonacci in O(log n).

Scenario 40: Merging Two Sorted Arrays Without Duplicates

You have two sorted arrays and want to merge them into one sorted array without duplicates.

Answer:

Since both arrays are already sorted, I can use a two-pointer technique to merge them efficiently. While merging, I'll skip duplicates.

Code Example:

```
import java.util.*;

public class MergeSortedArrays {
  public static void main(String[] args) {
    int[] arr1 = {1, 3, 5, 7};
    int[] arr2 = {2, 3, 6, 7, 8};

    List<Integer> merged = new ArrayList<>();
    int i = 0, j = 0;

    while (i < arr1.length && j < arr2.length) {
      if (arr1[i] < arr2[j]) {
        addIfNotPresent(merged, arr1[i++]);
      } else if (arr1[i] > arr2[j]) {
        addIfNotPresent(merged, arr2[j++]);
      } else {
        addIfNotPresent(merged, arr1[i]); // same element
        i++;
        j++;
      }
    }

    // Add remaining elements
    while (i < arr1.length) addIfNotPresent(merged, arr1[i++]);
    while (j < arr2.length) addIfNotPresent(merged, arr2[j++]);

    System.out.println("Merged array: " + merged);
  }
```

```
    private static void addIfNotPresent(List<Integer> list, int num) {
        if (list.isEmpty() || list.get(list.size() - 1) != num) {
            list.add(num);
        }
    }
}
```

Output:

Merged array: [1, 2, 3, 5, 6, 7, 8]

Reasoning to interviewer:

- Since arrays are sorted, two-pointer traversal ensures linear time O(n + m).
- Used a helper method addIfNotPresent() to avoid duplicates.
- Stored results in a List<Integer> for flexibility, but could convert to int[] later.

Alternative (Using Set, Simpler but less efficient):

```
Set<Integer> set = new TreeSet<>();
for (int num : arr1) set.add(num);
for (int num : arr2) set.add(num);

System.out.println("Merged array: " + set);
```

This works but takes extra log(n) time for insertion because of TreeSet.

Real-world extension:

- Used in database merges, log processing, and search index merging.
- Optimized merging like this is the core of merge sort and external sorting in big data systems.

Scenario 41: Keep Asking Until the Answer Is Correct (Quiz App)

You're building a quiz app. The system should keep asking the user for an answer until they type the correct one.

Answer:

Use a while(true) loop (or a do-while) and compare input using .equals() after normalizing spaces/case. Break when correct.

Code Example:

```
import java.util.Scanner;

public class QuizLoop {
   public static void main(String[] args) {
      final String CORRECT = "java";
      Scanner sc = new Scanner(System.in);

      while (true) {
         System.out.print("What language runs on the JVM? ");
         String ans = sc.nextLine();
         if (ans != null && ans.trim().equalsIgnoreCase(CORRECT)) {
            System.out.println("Correct!");
            break;
         }
         System.out.println("Try again.");
      }
   }
}
```

Output:

```
What language runs on the JVM? kotlin
Try again.
What language runs on the JVM?  JAVA
Correct!
```

Reasoning to interviewer:

- .equalsIgnoreCase() + trim() makes input forgiving.
- Infinite loop with an explicit break is clean for "retry-until-correct" flows.
- Reads with nextLine() to capture spaces and avoid scanner pitfalls.

Alternative (do-while):

```
String ans;
do {
   System.out.print("What language runs on the JVM? ");
   ans = sc.nextLine();
} while (ans == null || !ans.trim().equalsIgnoreCase(CORRECT));
System.out.println("Correct!");
```

Real-world extension:

- Add a retry limit and lockout after N attempts.
- Track attempts for analytics.
- Externalize correct answers to a config or DB for dynamic quizzes.

Scenario 42: Average of Daily Balances (Banking App)

In a banking app, you want to calculate the average of daily balances stored in an array.

Answer:

Accumulate balances using a long to avoid overflow, then divide by the count to get a double.

Code Example:

```
public class AverageBalance {
   public static double average(int[] balances) {
      if (balances == null || balances.length == 0) {
         throw new IllegalArgumentException("No balances to average");
      }
      long sum = 0L;
      for (int b : balances) sum += b;
      return sum / (double) balances.length;
   }

   public static void main(String[] args) {
      int[] balances = {1200, 1300, 1250, 1500};
      System.out.println("Average: " + average(balances));
   }
}
```

Output:

Average: 1312.5

Reasoning to interviewer:

- long for sum prevents overflow when arrays are large or values big.
- Returning double preserves precision.
- Throwing on empty input keeps downstream logic safe.

Alternative (Streams):

```
double avg = java.util.Arrays.stream(balances).average().orElse(0.0);
```

Real-world extension:

- Filter out weekends/holidays before averaging.
- Use a weighted average when days have different importance.
- Use BigDecimal for currency if precision rules require strict rounding.

Scenario 43: Longest Book Title (Library System)

For a library system, you want to find the longest book title in a list of titles.

Answer:

Iterate once, track the best-so-far by length, handle ties per business rule (e.g., keep the first).

Code Example:

```
import java.util.*;

public class LongestTitle {
   public static String longest(List<String> titles) {
      if (titles == null || titles.isEmpty()) return "";
      String best = "";
      for (String t : titles) {
         if (t != null && t.length() > best.length()) {
            best = t;
         }
      }
      return best;
   }

   public static void main(String[] args) {
      List<String> titles = Arrays.asList("Dune", "Clean Code", "Introduction to
Algorithms");
      System.out.println("Longest title: " + longest(titles));
   }
}
```

Output:

Longest title: Introduction to Algorithms

Reasoning to interviewer:

- Single pass O(n), constant extra space.
- Null-safe check so malformed data doesn't crash the flow.
- Tie-breaking is explicit (first longest wins).

Alternative (Streams):

```
String best = titles.stream()
   .filter(Objects::nonNull)
   .max(java.util.Comparator.comparingInt(String::length))
   .orElse("");
```

Real-world extension:

- Normalize by trimming; maybe compare by visible chars only.
- Localize: measuring grapheme length for multi-byte scripts.
- Add secondary sort (e.g., alphabetical) for ties.

Scenario 44: Multiplication Table (Ticket Pricing Rules)

In a ticket booking system, you need to generate a multiplication table (1 to 10) for pricing rules.

Answer:

Use nested loops: outer loop is the base, inner loop multiplies 1..10. Print in a grid.

Code Example:

```
public class MultiplicationTable {
   public static void main(String[] args) {
      for (int i = 1; i <= 10; i++) {
         StringBuilder row = new StringBuilder();
         for (int j = 1; j <= 10; j++) {
            row.append(String.format("%4d", i * j));
         }
         System.out.println(row);
      }
   }
}
```

Output (first 3 lines sample):

```
  1   2   3   4   5   6   7   8   9  10
  2   4   6   8  10  12  14  16  18  20
  3   6   9  12  15  18  21  24  27  30
...
```

Reasoning to interviewer:

- Formatting with String.format for neat alignment.
- Clear separation of concerns: compute vs. format.
- Easy to scale to N×M by changing loop bounds.

Alternative (Single base):

```
int base = 7;
for (int j = 1; j <= 10; j++) System.out.println(base + " x " + j + " = " + (base * j));
```

Real-world extension:

- Use these tables to derive tiered pricing or discount matrices.
- Export the grid to CSV for reporting.

Scenario 45: Count Vowels in a Sentence (Analytics App)

Your analytics app must count how many vowels appear in a given sentence.

Answer:

Traverse the string and tally a, e, i, o, u (case-insensitive). Optionally count each vowel separately.

Code Example:

```
import java.util.*;

public class VowelCount {
   private static final Set<Character> VOWELS =
         new HashSet<>(Arrays.asList('a','e','i','o','u'));

   public static int countVowels(String s) {
      if (s == null) return 0;
      int count = 0;
      for (char c : s.toLowerCase().toCharArray()) {
         if (VOWELS.contains(c)) count++;
      }
      return count;
   }

   public static void main(String[] args) {
      String sentence = "Data analysis is awesome!";
      System.out.println("Vowel count: " + countVowels(sentence));
   }
}
```

Output:

Vowel count: 10

Reasoning to interviewer:

- Lowercasing once avoids repeated case checks.
- Set lookup O(1) keeps it simple and fast.
- Returns 0 for null input—safe default.

Alternative (Per-vowel frequency):

```
int[] freq = new int[5]; // a e i o u
for (char c : s.toLowerCase().toCharArray()) {
   switch (c) {
      case 'a' -> freq[0]++;
      case 'e' -> freq[1]++;
      case 'i' -> freq[2]++;
      case 'o' -> freq[3]++;
      case 'u' -> freq[4]++;
   }
}
```

Real-world extension:

- Extend to all vowels in target languages.
- Use regex to strip punctuation/normalize whitespace before counting.

Scenario 46: Second Highest Score (Classroom Grading)

In a classroom grading system, you want to find the second highest score from an array of marks.

Answer:

Track max and second in one pass. Skip duplicates if the second highest must be strictly less than the highest.

Code Example:

```
import java.util.Arrays;

public class SecondHighest {
   public static Integer secondHighest(int[] arr) {
      if (arr == null || arr.length < 2) return null;
      Integer max = null, second = null;

      for (int n : arr) {
         if (max == null || n > max) {
            second = max;
            max = n;
         } else if (n != max && (second == null || n > second)) {
            second = n;
         }
      }
      return second; // null if all equal or not enough distinct values
   }

   public static void main(String[] args) {
      int[] scores = {88, 92, 75, 92, 85};
      System.out.println("Second highest: " + secondHighest(scores));
   }
}
```

Output:

Second highest: 88

Reasoning to interviewer:

- One pass, O(n), constant space.
- Distinguishes highest and strictly second highest.
- Returns null if not enough distinct values (explicit business behavior).

Alternative (Sorting):

```
java.util.Arrays.sort(scores);
int i = scores.length - 2;
while (i >= 0 && scores[i] == scores[i+1]) i--;
System.out.println(i >= 0 ? scores[i] : null);
```

Real-world extension:

- Useful for leaderboard logic, scholarship cutoffs, or top-N dashboards.
- Generalize to K-th largest via a min-heap of size K.

Scenario 47: Anagram Check (Word Puzzle)

For a word puzzle game, check if two words are anagrams (e.g., "listen" and "silent").

Answer:

Normalize strings (case, whitespace), then either sort and compare or count character frequencies.

Code Example (Sorting):

```
import java.util.Arrays;

public class AnagramCheck {
   public static boolean isAnagram(String a, String b) {
      if (a == null || b == null) return false;
      char[] x = a.replaceAll("\\s+", "").toLowerCase().toCharArray();
      char[] y = b.replaceAll("\\s+", "").toLowerCase().toCharArray();
      if (x.length != y.length) return false;
      Arrays.sort(x);
      Arrays.sort(y);
      return Arrays.equals(x, y);
   }

   public static void main(String[] args) {
      System.out.println(isAnagram("listen", "silent")); // true
      System.out.println(isAnagram("Java", "avaj "));    // true
      System.out.println(isAnagram("test", "text"));     // false
   }
}
```

Output:

```
true
true
false
```

Reasoning to interviewer:

- Sorting gives O(n log n) with small code.
- Normalizing (remove spaces/lowercase) avoids false negatives.
- For large alphabets, frequency counting can be O(n) time, O(1) space (fixed alphabet).

Alternative (Frequency map):

```
int[] freq = new int[26];
for (char c : x) freq[c-'a']++;
for (char c : y) if (--freq[c-'a'] < 0) return false;
return true;
```

Real-world extension:

- Case-insensitive search, scrabble helpers, and natural-language tooling.

Scenario 48: Emergency Cutoff on Temperature Stream (IoT)

In an IoT system, you receive a stream of temperature readings. You need to stop processing if the temperature crosses 100°C (emergency cutoff).

Answer:

Iterate readings, break immediately on cutoff, and log/alert.

Code Example:

```
public class TempCutoff {
   public static void main(String[] args) {
      int[] readings = {72, 76, 95, 99, 101, 88};

      for (int t : readings) {
         if (t > 100) {
            System.out.println("EMERGENCY: " + t + "°C – stopping processing!");
            break;
         }
         // normal processing
         System.out.println("Reading OK: " + t + "°C");
      }
   }
}
```

Output:

```
Reading OK: 72°C
Reading OK: 76°C
Reading OK: 95°C
Reading OK: 99°C
EMERGENCY: 101°C – stopping processing!
```

Reasoning to interviewer:

- break immediately stops further work to meet safety constraints.
- Keeps earlier valid data processed while halting at the first critical breach.
- Could pair with alerts/actuators (fan on/shutdown).

Alternative (Return from method when emergency requires full abort):

```
if (t > 100) { alert(); return; }
```

Real-world extension:

- Add hysteresis (require multiple high readings) to reduce false positives.
- Buffer recent readings for post-mortem analysis.

Scenario 49: Guess-the-Number Game

Generate a random number between 1 and 100 and let the user guess until they get it right.

Answer:

Use java.util.Random (or ThreadLocalRandom) and a loop that gives feedback "higher/lower".

Code Example:

```
import java.util.Random;
import java.util.Scanner;

public class GuessGame {
  public static void main(String[] args) {
    int target = new Random().nextInt(100) + 1; // 1..100
    Scanner sc = new Scanner(System.in);
    int attempts = 0;

    while (true) {
      System.out.print("Guess (1-100): ");
      int g = sc.nextInt();
      attempts++;

      if (g == target) {
        System.out.println("Correct in " + attempts + " attempts!");
        break;
      } else if (g < target) {
        System.out.println("Higher!");
      } else {
        System.out.println("Lower!");
      }
    }
  }
}
```

Output (sample):

```
Guess (1-100): 40
Higher!
Guess (1-100): 75
Lower!
Guess (1-100): 60
```

Correct in 3 attempts!

Reasoning to interviewer:

- nextInt(100)+1 generates 1..100 (inclusive).
- Simple loop with feedback keeps users engaged.
- Attempts counter adds UX polish.

Alternative (ThreadLocalRandom):

```
int target = java.util.concurrent.ThreadLocalRandom.current().nextInt(1, 101);
```

Real-world extension:

- Add attempt limits and hint tiers.
- Persist high scores; seed RNG for reproducible tests.

Scenario 50: Sum of Digits (Payroll Utility)

In a small payroll system, calculate the sum of digits of an employee's ID (e.g., 123 → 1+2+3 = 6).

Answer:

Peel off digits with % 10, accumulate, and reduce the number with / 10.

Code Example:

```
public class DigitSum {
   public static int sumDigits(int n) {
      n = Math.abs(n);
      int sum = 0;
      while (n > 0) {
         sum += n % 10; // last digit
         n /= 10;      // remove last digit
      }
      return sum;
   }

   public static void main(String[] args) {
      System.out.println(sumDigits(123));   // 6
      System.out.println(sumDigits(90705)); // 21
      System.out.println(sumDigits(-456));  // 15
   }
}
```

Output:

```
6
21
15
```

Reasoning to interviewer:

- Works for negatives via Math.abs.
- Loop is O(d) where d is number of digits—very fast.
- No string conversion, so memory-light and clear.

Alternative (String-based for readability):

```
int sum = 0;
for (char c : String.valueOf(Math.abs(n)).toCharArray()) sum += c - '0';
```

Real-world extension:

- Use as a checksum (e.g., simple validation for IDs).
- Combine with modulo arithmetic to create control digits.

Scenario 51: Second-Smallest Element (Distinct) in an Array

You need the second-smallest distinct value from an integer array. If it doesn't exist (all equal or <2 unique values), handle gracefully.

Answer:

Scan once while tracking min1 and min2. Update carefully to avoid counting duplicates.

Code Example:

```
public class SecondSmallest {
  public static Integer secondMin(int[] arr) {
    if (arr == null || arr.length < 2) return null;
    Integer min1 = null, min2 = null;

    for (int x : arr) {
      if (min1 == null || x < min1) {
        min2 = min1;
        min1 = x;
      } else if (x != min1 && (min2 == null || x < min2)) {
        min2 = x;
      }
    }
    return min2; // null if fewer than two distinct values
  }

  public static void main(String[] args) {
    int[] a = {7, 3, 3, 5, 9};
    System.out.println("Second smallest: " + secondMin(a)); // 5
  }
}
```

Output:

Second smallest: 5

Reasoning to interviewer:

- One pass O(n), constant space.
- Distinctness handled by x != min1.
- Returns null explicitly if there's no second distinct value.

Alternative (Sort then scan):

```
java.util.Arrays.sort(a);
Integer ans = null;
for (int i = 1; i < a.length; i++) if (a[i] != a[0]) { ans = a[i]; break; }
```

Real-world extension:

- Leaderboards needing runner-up rank.
- Scholarship/bonus logic (find 2nd best score).

Scenario 52: Rotate Array Right by k Positions (Cyclic Rotation)

Given an array, rotate it right by k steps: [1,2,3,4,5], k=2 → [4,5,1,2,3].

Answer:

Use the 3-reversals trick: reverse all, reverse first k, reverse last n-k.

Code Example:

```
import java.util.Arrays;

public class RotateArray {
   private static void reverse(int[] a, int l, int r) {
      while (l < r) {
         int t = a[l]; a[l] = a[r]; a[r] = t;
         l++; r--;
      }
   }

   public static void rotateRight(int[] a, int k) {
      if (a == null || a.length == 0) return;
      int n = a.length;
      k = ((k % n) + n) % n; // normalize
      reverse(a, 0, n - 1);
      reverse(a, 0, k - 1);
      reverse(a, k, n - 1);
   }

   public static void main(String[] args) {
      int[] a = {1,2,3,4,5};
      rotateRight(a, 2);
      System.out.println("Rotated: " + Arrays.toString(a));
   }
}
```

Output:

Rotated: [4, 5, 1, 2, 3]

Reasoning to interviewer:

- O(n) time, O(1) space.
- Reversal trick avoids extra arrays.

Alternative (extra array):

```
int[] b = new int[n];
for (int i = 0; i < n; i++) b[(i + k) % n] = a[i];
```

Real-world extension:

- Cyclic scheduling, circular buffers, round-robin resource assignment.

Scenario 53: Transpose a Matrix (m×n → n×m)

You want to transpose any rectangular matrix.

Answer:

Create a new matrix with swapped dimensions; copy [i][j] to [j][i].

Code Example:

```
import java.util.Arrays;

public class TransposeMatrix {
   public static int[][] transpose(int[][] M) {
      int m = M.length, n = M[0].length;
      int[][] T = new int[n][m];
      for (int i = 0; i < m; i++) {
         for (int j = 0; j < n; j++) {
            T[j][i] = M[i][j];
         }
      }
      return T;
   }

   public static void main(String[] args) {
      int[][] M = {{1,2,3},{4,5,6}};
      int[][] T = transpose(M);
      for (int[] row : T) System.out.println(Arrays.toString(row));
   }
}
```

Output:

```
[1, 4]
[2, 5]
[3, 6]
```

Reasoning to interviewer:

- O(mn) time, O(mn) extra space.
- Works for rectangular matrices (not only square).

Alternative (in-place for square):

Swap elements across the main diagonal with two indices (i<j).

Real-world extension:

- Rotations, linear algebra, CSV row/column transformations.

Scenario 54: Validate Phone Number Length & Digits

Accept only a 10-digit phone number (digits only). Reject others with a clear message.

Answer:

Check length and ensure all characters are digits.

Code Example:

```
import java.util.Scanner;

public class PhoneValidator {
   public static boolean isValidPhone(String s) {
      if (s == null || s.length() != 10) return false;
      for (int i = 0; i < s.length(); i++) {
         if (!Character.isDigit(s.charAt(i))) return false;
      }
      return true;
   }

   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      String phone = sc.nextLine().trim();
      System.out.println(isValidPhone(phone) ? "Valid" : "Invalid");
   }
}
```

Output:

9876543210 → Valid
98a6543210 → Invalid
12345 → Invalid

Reasoning to interviewer:

- Avoids regex overhead; character checks are clear and fast.
- Trim input to ignore leading/trailing spaces.

Alternative (Regex):

```
boolean ok = s != null && s.matches("\\d{10}");
```

Real-world extension:

- Add country codes; format or mask numbers; library validation for global formats (E.164).

Scenario 55: Running Average with a While Loop (Sentinel-Driven Input)

Keep reading numbers and print the running average until the user enters -1.

Answer:

Maintain sum and count. Update after each valid read; stop on sentinel.

Code Example:

```
import java.util.Scanner;

public class RunningAverage {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      long sum = 0; int count = 0;

      while (true) {
         System.out.print("Enter number (-1 to stop): ");
         int x = sc.nextInt();
         if (x == -1) break;
         sum += x; count++;
         System.out.printf("Running avg: %.2f%n", sum / (double) count);
      }
      System.out.println("Final avg: " + (count == 0 ? 0 : sum / (double) count));
   }
}
```

Output (sample):

```
Enter number (-1 to stop): 10
Running avg: 10.00
Enter number (-1 to stop): 20
Running avg: 15.00
Enter number (-1 to stop): -1
Final avg: 15.0
```

Reasoning to interviewer:

- While loop fits unknown count.
- long sum avoids overflow; prints with 2 decimal places.

Alternative:

Use nextLine() + parse to handle non-numeric input safely.

Real-world extension:

- Sensor streams, live analytics, rolling averages (windowed).

Scenario 56: Word Frequency (Count Duplicate Words in a Sentence)

Count occurrences of each word, ignoring case and punctuation.

Answer:

Normalize (lowercase, trim, remove punctuation), then use a HashMap.

Code Example:

```
import java.util.*;

public class WordFrequency {
   public static Map<String,Integer> freq(String text) {
      Map<String,Integer> map = new HashMap<>();
      if (text == null || text.isBlank()) return map;

      String[] words = text.toLowerCase()
            .replaceAll("[^a-z0-9\\s]", " ")
            .trim()
            .split("\\s+");

      for (String w : words) {
         if (w.isEmpty()) continue;
         map.put(w, map.getOrDefault(w, 0) + 1);
      }
      return map;
   }

   public static void main(String[] args) {
      System.out.println(freq("Java, java! is great; JAVA is popular."));
   }
}
```

Output:

{java=3, is=2, great=1, popular=1}

Reasoning to interviewer:

- Normalization prevents punctuation/case issues.
- getOrDefault simplifies counting.

Alternative:

Use LinkedHashMap to preserve first-seen order, or TreeMap for sorted keys.

Real-world extension:

- Search suggestions, tag clouds, spam detection.

Scenario 57: Remove Vowels from a String

Given a string, remove vowels to produce a consonant-only version.

Answer:

Use a StringBuilder and skip characters that match a vowel set (case-insensitive).

Code Example:

```
import java.util.Set;
import java.util.HashSet;
import java.util.Arrays;

public class RemoveVowels {
   private static final Set<Character> VOWELS =
      new HashSet<>(Arrays.asList('a','e','i','o','u'));

   public static String stripVowels(String s) {
      if (s == null) return null;
      StringBuilder sb = new StringBuilder(s.length());
      for (char c : s.toCharArray()) {
         if (!VOWELS.contains(Character.toLowerCase(c))) sb.append(c);
      }
      return sb.toString();
   }

   public static void main(String[] args) {
      System.out.println(stripVowels("Interview Ready")); // ntrvw Rdy
   }
}
```

Output:

ntrvw Rdy

Reasoning to interviewer:

- StringBuilder is efficient for repeated appends.
- Lowercase once per char for case-insensitive test.

Alternative (Regex):

```
s.replaceAll("(?i)[aeiou]", "")
```

Real-world extension:

- URL slugs, compact identifiers, playful UIs.

Scenario 58: Sum Primary & Secondary Diagonals of a Square Matrix

Compute the sum of both diagonals; avoid double-counting the center for odd n.

Answer:

Loop i=0..n-1: add a[i][i] and a[i][n-1-i]. If i == n-1-i, subtract one center once.

Code Example:

```
public class DiagonalSum {
   public static int diagonalSum(int[][] a) {
      int n = a.length, sum = 0;
      for (int i = 0; i < n; i++) {
         sum += a[i][i];
         sum += a[i][n - 1 - i];
      }
      if (n % 2 == 1) {
         sum -= a[n/2][n/2]; // center counted twice
      }
      return sum;
   }

   public static void main(String[] args) {
      int[][] m = {
         {1,2,3},
         {4,5,6},
         {7,8,9}
      };
      System.out.println(diagonalSum(m)); // 25
   }
}
```

Output:

25

Reasoning to interviewer:

- O(n) time, O(1) space for n×n matrix.
- Correctly handles odd dimension center.

Alternative:

Return both sums separately if the use-case needs them individually.

Real-world extension:

- Image processing kernels, cost matrices, puzzle/board games.

Scenario 59: Intersection of Two Sorted Arrays (Unique)

Find unique numbers present in both sorted arrays.

Answer:

Use two pointers; when equal, add once and advance both; skip duplicates.

Code Example:

```
import java.util.*;

public class SortedIntersection {
   public static List<Integer> intersect(int[] a, int[] b) {
      List<Integer> res = new ArrayList<>();
      int i = 0, j = 0;
      Integer last = null;

      while (i < a.length && j < b.length) {
         if (a[i] < b[j]) i++;
         else if (a[i] > b[j]) j++;
         else {
            if (last == null || last != a[i]) {
               res.add(a[i]);
               last = a[i];
            }
            i++; j++;
         }
      }
      return res;
   }

   public static void main(String[] args) {
      int[] x = {1,2,2,3,5,7};
      int[] y = {2,2,4,5,6,7,7};
      System.out.println("Intersection: " + intersect(x,y)); // [2,5,7]
   }
}
```

Output:

Intersection: [2, 5, 7]

Reasoning to interviewer:

- O(n + m) time, O(1) extra besides output.
- Maintains uniqueness by tracking last.

Alternative (Set-based, unsorted):
Use HashSet for one array, probe with the other, gather unique results in LinkedHashSet.

Real-world extension:

- Common customers between datasets, overlapping tags/interests.

Scenario 60: Validate PIN (Exactly 4 or 6 Digits)

Accept only numeric PINs of length 4 or 6.

Answer:

Length check + all digits. Make error messaging explicit.

Code Example:

```
public class PinValidator {
    public static boolean isValidPin(String s) {
        if (s == null) return false;
        int len = s.length();
        if (len != 4 && len != 6) return false;
        for (int i = 0; i < len; i++) {
            if (!Character.isDigit(s.charAt(i))) return false;
        }
        return true;
    }

    public static void main(String[] args) {
        System.out.println(isValidPin("1234"));   // true
        System.out.println(isValidPin("12a4"));   // false
        System.out.println(isValidPin("123456")); // true
        System.out.println(isValidPin("12345"));  // false
    }
}
```

Output:

```
true
false
true
false
```

Reasoning to interviewer:

- Clear, fast digit check; no regex necessary.
- Defensive null handling.

Alternative (Regex):

```
boolean ok = s != null && s.matches("^\\d{4}$|^\\d{6}$");
```

Real-world extension:

- Add retry limits, lockouts, and server-side validation.